



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

DIPARTIMENTO DI MATEMATICA
LAUREA TRIENNALE IN INFORMATICA

PROGETTO DI BASI DI DATI

**Basi di Dati di un Sistema Informativo per la
Gestione delle ultime venti stagioni regolari di
NBA**

Bruno Grimaldi - 2048056
Riccardo Baldin - 2075548

1 ABSTRACT

Si vuole creare una base dati per rappresentare le partite disputate e le statistiche dei giocatori e delle squadre nel campionato NBA.

Ogni membro di una squadra è rappresentato come un'entità distinta, suddivisa in giocatori e staff tecnico, con caratteristiche specifiche a seconda del ruolo.

I giocatori prendono parte alle partite ufficiali del campionato, organizzate in stagioni, durante le quali producono statistiche individuali come punti, rimbalzi e assist.

Le partite si svolgono in stadi associati alle squadre di casa e prevedono sempre un vincitore.

Le squadre appartengono a una conferenza e ogni componente della squadra, sia esso giocatore o staff, stipula un contratto a durata determinata con la squadra stessa, che ne registra lo stipendio e il periodo di validità.

Le prestazioni dei giocatori nel corso delle stagioni possono essere premiate con riconoscimenti ufficiali.

Il sistema permette così di tenere traccia dell'evoluzione delle squadre, dei contratti, delle stagioni e dei risultati ottenuti da ciascun atleta nel tempo.

2 ANALISI DEI REQUISITI

Si crea un database per tenere traccia delle partite effettuate dai giocatori delle varie squadre, in diverse stagioni, di seguito i requisiti fondamentali della base di dati.

- **Membro Squadra.** Ogni persona interna alle squadre dell'NBA è identificata da un codice univoco e contiene le seguenti informazioni:

- Un codice identificativo univoco
- Nome
- Cognome
- Data di nascita
- **Giocatore.** Oltre alle informazioni generali, i giocatori hanno anche le seguenti:
 - * Nazionalità
 - * Peso
 - * Altezza
 - * Ruolo in campo

- * Qualifica
 - **Staff.** Oltre alle informazioni generali, i membri dello staff hanno solo la qualifica, ma non le specifiche proprie dei giocatori:
- * Qualifica
- **Squadra.** Le squadre sono popolate dai membri e hanno bisogno di essere differenziate in base a:
 - Nome
 - Conferenza in cui gioca la Squadra
- **Stadio.** Ogni squadra ha uno stadio di appartenenza in cui gioca le partite in casa, per ogni stato sono presenti:
 - Nome
 - Città
 - Capienza
- **Partita.** Delle partite giocate durante la stagione regolare si sanno le seguenti informazioni:
 - Data in cui si svolge la partita
 - Nome della squadra che gioca in casa
 - Nome della squadra che gioca fuori casa
 - Punteggio della squadra di casa
 - Punteggio della casa ospite
 - Vincitore della partita
- **Analisi Partita.** Per ogni giocatore per vengono salvate le statistiche riguardo alle partite che gioca, esse sono formate da:
 - Punti realizzati nella partita
 - Rimbalzi presi
 - Assist fatti
- **Stagione.** Una stagione, periodo temporale in cui si raggruppano un insieme di partite, è caratterizzata da:
 - Numero identificativo
 - Data di inizio
 - data di fine
- **Premio.** Ogni anno ai giocatori più meritevoli vengono assegnati dei premi di cui bisogna conoscere:
 - Nome

3 PROGETTAZIONE CONCETTUALE

3.1 Lista Entità

Ogni entità del database viene rappresentata dalle seguenti tabelle; tutti gli attributi sono NOT NULL tranne quelli specificati. Alcuni campi sono di tipo *enum*, creato appositamente per quegli attributi.

- **Giocatore:** rappresenta un giocatore di una squadra che scende in campo
 - ID_Membro INT
 - Nome VARCHAR
 - Cognome VARCHAR
 - Data_Nascita DATE
 - Nazionalità VARCHAR
 - Peso INT
 - Altezza INT
 - Ruolo RUOLO_ENUM
- **Staff:** rappresenta un membro di uno staff della squadra
 - ID_Membro INT
 - Nome VARCHAR
 - Cognome VARCHAR
 - Data_Nascita DATE
 - Qualifica QUALIFICA_ENUM
- **Stadio:** rappresenta uno stadio in cui gioca una squadra quando svolge partite in casa
 - Nome_Stadio VARCHAR
 - Città VARCHAR
 - Capacità INT NULL
- **Squadra:** rappresenta una squadra che partecipa al campionato di NBA
 - Nome_Squadra VARCHAR
 - Conferenza CONFERENZA_ENUM
 - Nome_Stadio VARCHAR
- **Contratto:** rappresenta un vincolo legale tra un Membro_Squadra e una Squadra

- ID_Membro INT
- Nome_Squadra VARCHAR
- Inizio_Contratto DATE
- Fine_Contratto DATE
- Stipendio INT
- **Stagione:** rappresenta un anno non solare durante il quale vengono svolte delle partite
 - Numero_Stagione INT
 - Data_Inizio DATE
 - Data_Fine DATE
- **Partita:** rappresenta una partita disputata tra due squadre
 - Data_Partita DATE
 - Squadra_Casa VARCHAR
 - Squadra_Ospiti VARCHAR
 - Punteggio_Casa INT
 - Punteggio_Ospiti INT
 - Numero_Stagione INT
 - Vincitore BOOL NULL
- **Analisi_Partita:** rappresenta le statistiche generate da un Giocatore durante una Partita
 - ID_Membro INT
 - Data_Partita DATE
 - Squadra_Casa VARCHAR
 - Punti INT
 - Rimbalzi INT
 - Assist INT
- **Premio:** rappresenta un premio assegnato a un giocatore alla fine di una stagione
 - Nome_Premio NOME_PREMIO
 - ID_Membro INT
 - Numero_Stagione INT

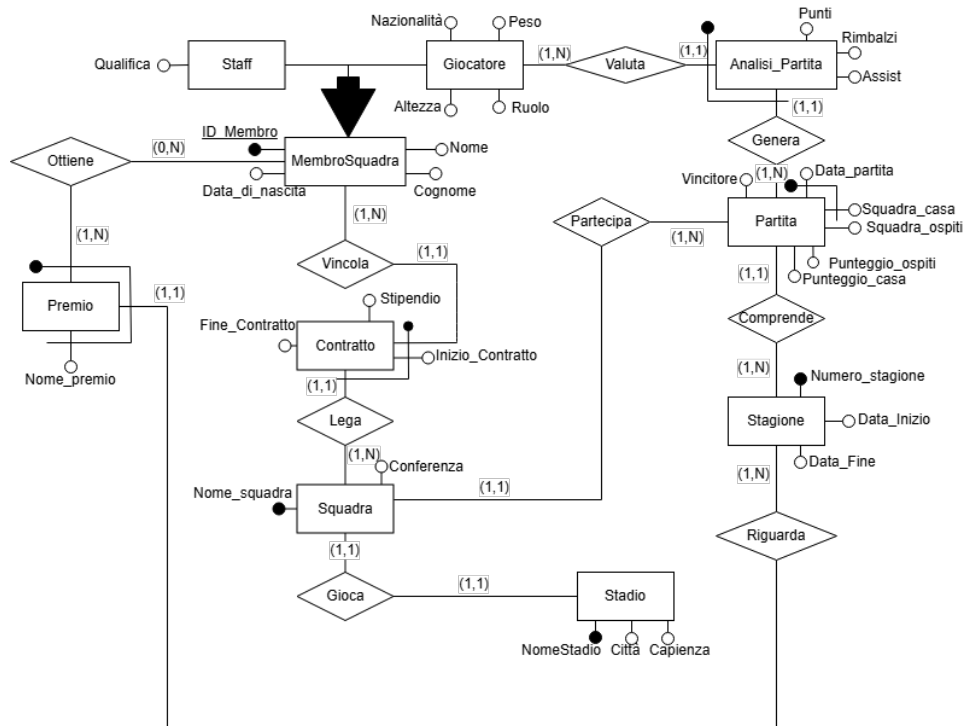
3.2 Lista Relazioni

- **Contratto-Membro__Squadra:** vincola (1-N)
 - Un contratto vincola un solo membro di una squadra
 - Un membro di una squadra può avere firmato più contratti
- **Contratto-Squadra:** lega (1-N)
 - Un contratto è riferito ad una sola squadra
 - Una squadra stipula molteplici contratti per più giocatori ogni anno
- **Squadra-Partita:** partecipa (1-N)
 - Una squadra partecipa ad una sola partita alla volta
 - Una partita coinvolge più di una squadra (due) per volta
- **Squadra-Stadio:** gioca (1-1)
 - Una squadra possiede il suo stadio dove gioca in casa le partite
 - Uno stadio ospita sempre una sola squadra in casa
- **Premio-Stagione:** riguarda (1-N)
 - Un premio viene assegnato solo una volta per stagione
 - In una stagione vengono assegnati molteplici premi
- **Premio-Membro__Squadra:** ottiene(N-M)
 - Un premio può essere assegnato a più giocatori contemporaneamente
 - Un giocatore può ricevere più premi nel corso della sua vita
- **Analisi__Partita-Membro__Squadra:** valuta(1-N)
 - Una analisi partita è riferita ad un solo giocatore del quale specifica le statistiche
 - Un giocatore è valutato in tante analisi quante sono le partite che gioca
- **Analisi__Partita-Partita:** genera(1-N)
 - Una analisi partita è riferita ad una sola partita da analizzare
 - Una partita genera tante analisi quante sono le persone che la giocano
- **Partita-Stagione:** comprende(1-N)
 - Una partita viene catalogata e giocata in una singola stagione
 - Una stagione regolare contiene 1230 partite

3.3 Lista Generalizzazioni

Membro_Squadra è una generalizzazione totale ed esclusiva di **Giocatore** e **Staff**.

3.4 Schema E-R



4 PROGETTAZIONE LOGICA

4.1 Analisi delle ridondanze

L'attributo Vincitore dell'entità **Partita** memorizza il vincitore di una partita calcolando i risultati memorizzati in Punteggio_Casa e Punteggio_Ospiti, andando a creare una ridondanza, essendo possibile ottenerlo direttamente nelle operazioni che lo richiedono senza che venga memorizzato.

Questo attributo viene modificato solo se avvengono cambiamenti ai punteggi in caso di specifici eventi che vanno a modificare i risultati di una partita a posteriori, come l'annullamento della stessa, e viene visualizzato quando si desidera ottenere statistiche in tempo reale, ad esempio su una dashboard web dove i tifosi possono accedere per controllare statistiche aggiornate ad ogni partita della loro squadra del cuore.

Tutto questo si riassume in:

- **Operazione 1**(50 inserimenti a settimana): memorizzare una nuova partita con relativo vincitore.

- **Operazione 2**(2000 letture a settimana): visualizzare il vincitore o estrapolarne il dato per calcolare ulteriori statistiche.

Concetto	Costrutto	Volume
Partita	Entità	50

La seguente analisi serve a stabilire se serve o meno mantenere l'attributo ridondante Vincitore in **Partita**, si ricorda che ogni operazione di scrittura ha peso pari a 2 accessi e che il tempo per il calcolo dei dati è trascurabile

Con Ridondanza

- **Operazione 1**

Concetto	Costrutto	Accessi	Tipo
Partita(Punteggio Casa)	Entità	1	Scrittura
Partita(Punteggio Ospiti)	Entità	1	Scrittura
Partita(Vincitore)	Entità	1	Scrittura

Costo:

Volume * scrittura dei due punteggi e di vincitore $\rightarrow 50 * (2*3) = 300$

- **Operazione 2**

Concetto	Costrutto	Accessi	Tipo
Partita(Vincitore)	Entità	1	Lettura

Costo:

Lettura di Vincitore * #accessi settimanali $\rightarrow 1 * 2000 = 2000$

Costo totale con ridondanza: Operazione1 + Operazione2 = 2300

Senza Ridondanza

- **Operazione 1**

Concetto	Costrutto	Accessi	Tipo
Partita(Punteggio Casa)	Entità	1	Scrittura
Partita(Punteggio Ospiti)	Entità	1	Scrittura

Costo:

Volume * scrittura dei due punteggi e di vincitore $\rightarrow 50 * (2*2) = 200$

• Operazione 2

Concetto	Costrutto	Accessi	Tipo
Partita(Punteggio Casa)	Entità	1	Lettura
Partita(Punteggio Ospiti)	Entità	1	Lettura

Costo:

Lettura dei 2 risultati * #accessi settimanali $\rightarrow 2 * 2000 = 4000$

Costo totale senza ridondanza: Operazione1 + Operazione2 = 4200

Si può quindi notare che mantenere la ridondanza si dimostra vantaggioso perché, nel caso verosimile di molteplici accessi al giorno, va a ridurre il costo totale di circa 45%

4.2 Eliminazione delle Generalizzazioni

Membro_Squadra: è una generalizzazione totale ed esclusiva, legata alle entità Giocatore e Staff. La generalizzazione viene eliminata accorpando le entità figlie nell'entità padre, rendendo possibile l'inserimento del valore NULL in alcuni attributi. Viene aggiunto "Giocatore" nei valori possibili dell'elenco "enum" della qualifica. L'entità quindi diventa:

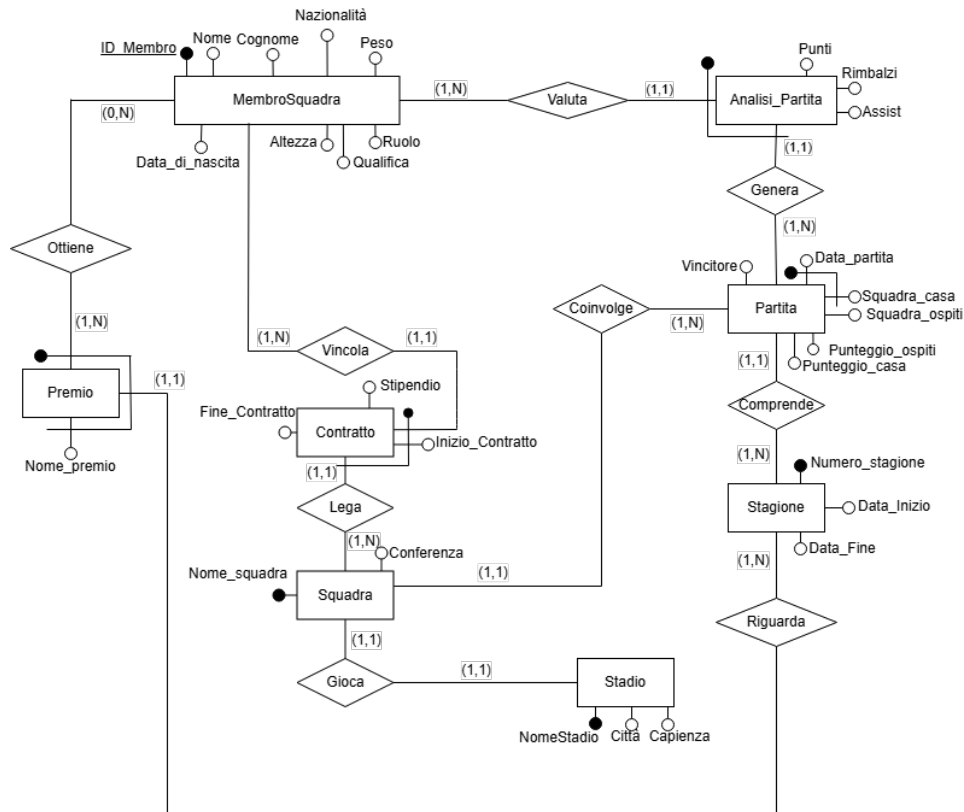
- **Membro_Squadra:** rappresenta un giocatore o un membro dello staff di una squadra
 - ID_Membro INT NOT NULL
 - Nome VARCHAR NOT NULL
 - Cognome VARCHAR NOT NULL
 - Data_Di_Nascita DATE
 - Nazionalità VARCHAR
 - Peso INT
 - Altezza INT
 - Qualifica ENUM NOT NULL
 - Ruolo ENUM

4.3 Vincoli non esprimibili nel diagramma E-R

Sono stati creati dei trigger e delle funzioni per controllare l'inserimento e la modifica delle tuple in:

- **Contratto:** all'inserimento di un nuovo contratto per un Membro_Squadra si controlla che il suddetto non sia già vincolato ad una società tramite un altro contratto attivo. Si controlla quindi nello specifico che la data di inizio non sia compresa nel periodo di ogni altro contratto del giocatore e che la data di inizio sia precedente alla data di fine contratto.
- **Premio:** all'inserimento di un nuovo premio assegnato ad un Membro_Squadra si controlla che:
 - I premi unici (ad esempio "MVP") vengano assegnati esclusivamente una volta per stagione.
 - I premi multipli (ad esempio "All-NBA Team") vengano assegnati ad un massimo di 5 persone per stagione.
- **Partita:** L'attributo Vincitore è un valore booleano che si auto-genera in base ai punteggi della squadra di casa e della squadra ospite al momento dell'inserimento di una tupla; nel caso i punteggi venissero aggiornati a causa di irregolarità sportive o arbitrali verificate a partita finita, Vincitore si aggiornerebbe automaticamente.
- **Membro_Squadra:** all'inserimento di un nuovo Membro_Squadra, nel caso la sua qualifica fosse "Giocatore", gli attributi "Peso", "Altezza", "Nazionalità" e "Ruolo" non potrebbero essere lasciati vuoti, nel caso contrario invece il database non accetta informazioni riguardanti gli attributi appena citati.

4.4 Diagramma E-R ristrutturato



4.5 Descrizione schema relazionale

- **Membro_Squadra**(ID_Membro, Nome, Cognome, Data_Di_Nascita, Nazionalità, Peso, Altezza, Qualifica, Ruolo)
- **Stadio**(Nome_Stadio, Città, Capienza)
- **Squadra**(Nome_Squadra, Conferenza, Nome_Stadio)
 - Squadra.Nome_Stadio → Stadio.Nome_Stadio
- **Contratto**(ID_Membro, Nome_Squadra, Inizio_Contratto, Fine_Contratto, Stipendio)
 - Contratto.ID_Membro → Membro_Squadra.ID_Membro
 - Contratto.Nome_Squadra → Squadra.Nome_Squadra
- **Stagione**(Numero_stagione, Data_inizio, Data_fine)
- **Partita**(Data_Partita, Squadra_casa, Squadra_ospiti, Punteggio_casa, Punteggio_ospiti, Vincitore, Numero_stagione)
 - Partita.Squadra_casa → Squadra.Nome_Squadra

- Partita.Squadra Ospiti → Squadra.Nome_Squadra
- Partita.Numero_stagione → Stagione.Numero_stagione
- **Analisi_Partita**(ID_Membro, Data_partita, Squadra_casa, Punti, Rimbalzi, Assist)
 - Analisi_Partita.ID_Membro → Membro_Squadra.ID_Membro
 - Analisi_Partita.Data_partita → Partita.Data_partita
 - Analisi_Partita.Squadra_casa → Partita.Squadra_casa
- **Premio**(Nome_premio, Numero_stagione, ID_Membro)
 - Premio.Numero_stagione → Stagione.Numero_stagione
 - Premio.ID_Membro → Membro_Squadra.ID_Membro

5 IMPLEMENTAZIONE IN POSTGRESQL E DEFINIZIONE DELLE QUERY

Di seguito vengono presentate le query mostrando il codice SQL e l'output generato tramite pgAdmin4 per testare il Database sopra presentato e viene discussa la scelta dell'indice proposto

5.1 Definizione delle Query

Query 1: Classifica delle prime 5 squadre per rapporto tra vittorie e partite giocate in una stagione, in base all'anno inserito dall'utente:

```
WITH Statistiche AS (
  SELECT
    s.Nome_Squadra,
    SUM(CASE
      WHEN (p.Vincitore = TRUE
      AND p.Squadra_Casa = s.Nome_Squadra)
      OR (p.Vincitore = FALSE
      AND p.Squadra_Ospiti = s.Nome_Squadra)
      THEN 1 ELSE 0
    END) AS Vittorie,
    SUM(CASE
      WHEN (p.Vincitore = FALSE
      AND p.Squadra_Casa = s.Nome_Squadra)
      OR (p.Vincitore = TRUE
      AND p.Squadra_Ospiti = s.Nome_Squadra)
      THEN 1 ELSE 0
    END) AS Sconfitte
```

```

        FROM Partita p
        JOIN Squadra s ON s.Nome_Squadra = p.Squadra_Casa OR
        s.Nome_Squadra = p.Squadra_Ospiti
        WHERE p.Numero_Stagione = 2024
        GROUP BY s.Nome_Squadra
    )
    SELECT
        Nome_Squadra,
        CASE
            WHEN (Vittorie + Sconfitte) = 0 THEN NULL
            ELSE ROUND(
                CAST(Vittorie AS numeric) / (Vittorie + Sconfitte),
                3)
        END AS Rapporto_Vittorie_Partite,
        Vittorie,
        Sconfitte
    FROM Statistiche
    ORDER BY Rapporto_Vittorie_Partite DESC
    LIMIT 5;

```

	nome_squadra [PK] character varying (32)	rapporto_vittorie_partite numeric	vittorie bigint	sconfitte bigint
1	Oklahoma City Thunder	0.819	68	15
2	Cleveland Cavaliers	0.780	64	18
3	Boston Celtics	0.744	61	21
4	Houston Rockets	0.634	52	30
5	New York Knicks	0.622	51	31

Query 2: Giocatore (o giocatori in caso di parità) che ha collezionato più premi in tutta la carriera:

```

SELECT ms.Nome, ms.Cognome, COUNT(*) AS Numero_Premi
FROM Premio pr
JOIN Membro_Squadra ms ON pr.ID_Membro = ms.ID_Membro
WHERE ms.Qualifica = 'Giocatore'
GROUP BY ms.ID_Membro, ms.Nome, ms.Cognome
HAVING COUNT(*) = (
    SELECT MAX(Numero_Premi) FROM (
        SELECT COUNT(*) AS Numero_Premi
        FROM Premio pr
        JOIN Membro_Squadra ms ON pr.ID_Membro = ms.ID_Membro
        WHERE ms.Qualifica = 'Giocatore'
        GROUP BY ms.ID_Membro
    ) AS Premi_Giocatori
);

```

	nome character varying (32)	cognome character varying (32)	numero_premi bigint
1	Rudy	Gobert	5
2	Bam	Adebayo	5

Query 3: Giocatore (o giocatori in caso di parità) che ha collezionato più premi dello stesso tipo (scelto da utente tra i disponibili)

```
SELECT ms.Nome, ms.Cognome, COUNT(*) AS Premio_individuale
FROM Premio pr
JOIN Membro_Squadra ms ON pr.ID_Membro = ms.ID_Membro
WHERE ms.Qualifica = 'Giocatore'
AND pr.Nome_Premio = 'MVP'
GROUP BY ms.ID_Membro, ms.Nome, ms.Cognome
HAVING COUNT(*) = (
    SELECT MAX(Premio_individuale) FROM (
        SELECT COUNT(*) AS Premio_individuale
        FROM Premio pr
        JOIN Membro_Squadra ms ON pr.ID_Membro = ms.ID_Membro
        WHERE ms.Qualifica = 'Giocatore'
        AND pr.Nome_Premio = 'MVP'
        GROUP BY ms.ID_Membro
    ) AS Premio_individuale
);
```

	nome character varying (32)	cognome character varying (32)	premio_individuale bigint
1	Nikola	Jokic	2

Query 4: Squadre che hanno più giocatori con N premi (numero scelto da utente)

```
WITH Giocatori_Con_X_Premi AS (
    SELECT ID_Membro
    FROM Premio
    GROUP BY ID_Membro
    HAVING COUNT(*) >= 0
),
Giocatori_Squadre AS (
    SELECT DISTINCT c.Nome_Squadra, c.ID_Membro
    FROM Contratto c
    JOIN Giocatori_Con_X_Premi g ON g.ID_Membro = c.ID_Membro
),
```

```

Squadre_Con_Conteggio AS (
    SELECT gs.Nome_Squadra, COUNT(*) AS Giocatori_Con_Premi
    FROM Giocatori_Squadre gs
    GROUP BY gs.Nome_Squadra
),

Max_Valore AS (
    SELECT MAX(Giocatori_Con_Premi) AS Max_Premi
    FROM Squadre_Con_Conteggio
)

SELECT s.Nome_Squadra, s.Giocatori_Con_Premi
FROM Squadre_Con_Conteggio s
JOIN Max_Valore m ON s.Giocatori_Con_Premi = m.Max_Premi;

```

	nome_squadra character varying (32) 	giocatori_con_premi bigint 
1	Sacramento Kings	2
2	Utah Jazz	2
3	Milwaukee Bucks	2
4	Miami Heat	2
5	Brooklyn Nets	2
6	Golden State Warriors	2
7	Atlanta Hawks	2

Query 5: Giocatore che ha firmato il maggior numero di contratti negli ultimi vent'anni, e numero di squadre con le quali ha firmato

```

SELECT ms.Nome, ms.Cognome, COUNT(*) AS Numero_Contratti,
COUNT(DISTINCT c.Nome_Squadra) AS Squadre_Diverse
FROM Contratto c
JOIN Membro_Squadra ms ON c.ID_Membro = ms.ID_Membro
WHERE ms.Qualifica = 'Giocatore'
GROUP BY ms.ID_Membro, ms.Nome, ms.Cognome
HAVING COUNT(*) = (
    SELECT MAX(Numero_Contratti) FROM (
        SELECT COUNT(*) AS Numero_Contratti

```

```

        FROM Contratto c JOIN Membro_Squadra ms
        ON c.ID_Membro = ms.ID_Membro
        WHERE ms.Qualifica = 'Giocatore'
              GROUP BY ms.ID_Membro
    ) AS Numero_Contratti
);

```

	nome character varying (32) 🔒	cognome character varying (32) 🔒	numero_contratti bigint 🔒	squadre_diverse bigint 🔒
1	LeBron	James	3	1

Query 6: Tra due squadre, scelte dall'utente, restituire la data della partita degli ultimi venti anni con più grande distacco di punti tra le due squadre e lo stadio in cui è stata giocata

```

SELECT
    p.Data_Partita,
    ABS(p.Punteggio_Casa - p.Punteggio_Ospiti) AS Distacco,
    CASE
        WHEN p.Punteggio_Casa > p.Punteggio_Ospiti THEN p.Squadra_Casa
        ELSE p.Squadra_Ospiti
    END AS Squadra_Vincente,
    sq.Nome_Stadio
FROM Partita p
JOIN Squadra sq ON p.Squadra_Casa = sq.Nome_Squadra
WHERE (
    (p.Squadra_Casa = 'Los Angeles Lakers'
     AND p.Squadra_Ospiti = 'Boston Celtics')
    OR (p.Squadra_Casa = 'Boston Celtics'
     AND p.Squadra_Ospiti = 'Los Angeles Lakers')
)
AND p.Data_Partita >= CURRENT_DATE - INTERVAL '20 years'
AND ABS(p.Punteggio_Casa - p.Punteggio_Ospiti) = (
    SELECT MAX(ABS(p2.Punteggio_Casa - p2.Punteggio_Ospiti))
    FROM Partita p2
    WHERE (
        (p2.Squadra_Casa = 'Los Angeles Lakers'
         AND p2.Squadra_Ospiti = 'Boston Celtics')
        OR (p2.Squadra_Casa = 'Boston Celtics'
         AND p2.Squadra_Ospiti = 'Los Angeles Lakers')
    )
    AND p2.Data_Partita >= CURRENT_DATE - INTERVAL '20 years'
);

```


	data_partita date	distacco integer	squadra_vincente character varying (32)	nome_stadio character varying (32)
1	2025-01-24	21	Los Angeles Lakers	Crypto.com Arena

Query 7: Squadre con giocatori che nell'anno selezionato dall'utente, hanno collezionato più triple-doppie (= punti ≥ 10 , rimbalzi ≥ 10 e assist ≥ 10). Stampa anche il numero di triple-doppie e il giocatore che ne ha collezionate di più per ogni squadra

```

WITH TripleDoppie AS (
    SELECT
        ap.ID_Membro,
        p.Numero_Stagione,
        c.Nome_Squadra,
        COUNT(*) AS Num_Triple_Doppie
    FROM Analisi_Partita ap
    JOIN Partita p
        ON ap.Data_Partita = p.Data_Partita
        AND ap.Squadra_Casa = p.Squadra_Casa
    JOIN Contratto c
        ON ap.ID_Membro = c.ID_Membro
        AND p.Data_Partita BETWEEN c.Inizio_Contratto
        AND c.Fine_Contratto
    WHERE
        ((ap.Punti >= 10)::int +
         (ap.Rimbalzi >= 10)::int +
         (ap.Assist >= 10)::int) >= 3
        AND p.Numero_Stagione = 2023
    GROUP BY ap.ID_Membro, p.Numero_Stagione, c.Nome_Squadra
),

SquadraTriple AS (
    SELECT
        Nome_Squadra,
        SUM(Num_Triple_Doppie) AS Totale_Triple_Doppie
    FROM TripleDoppie
    GROUP BY Nome_Squadra
),

TopGiocatori AS (
    SELECT
        td.Nome_Squadra,
        ms.Nome || ' ' || ms.Cognome AS Giocatore,
        td.Num_Triple_Doppie,
        ROW_NUMBER() OVER (PARTITION BY td.Nome_Squadra

```

```

        ORDER BY td.Num_Triple_Doppie DESC) AS rn
    FROM TripleDoppie td
    JOIN Membro_Squadra ms ON td.ID_Membro = ms.ID_Membro
)

SELECT
    st.Nome_Squadra,
    st.Totale_Triple_Doppie,
    tg.Giocatore,
    tg.Num_Triple_Doppie AS Triple_Doppie_Giocatore
FROM SquadraTriple st
JOIN TopGiocatori tg
    ON st.Nome_Squadra = tg.Nome_Squadra AND tg.rn = 1
ORDER BY st.Totale_Triple_Doppie DESC

```

	nome_squadra character varying (32)	totale_triple_doppie numeric	giocatore text	triple_doppie_giocatore bigint
1	Sacramento Kings	4	Dennis Schroder	4
2	San Antonio Spurs	1	Bradley Beal	1

Query 8: Classifica degli N giocatori (numero scelto dall'utente) che hanno accumulato più punti in carriera specificando in quante partite hanno giocato

```

SELECT ms.Nome, ms.Cognome, COUNT(DISTINCT ap.Data_Partita)
1AS Partite_Giocate, SUM(ap.Punti) AS Totale_Punti
FROM Analisi_Partita ap
JOIN Membro_Squadra ms ON ap.ID_Membro = ms.ID_Membro
WHERE ms.Qualifica = 'Giocatore'
GROUP BY ms.ID_Membro, ms.Nome, ms.Cognome
ORDER BY Totale_Punti DESC, Partite_Giocate ASC
LIMIT 2;

```

	nome character varying (32)	cognome character varying (32)	partite_giocate bigint	totale_punti bigint
1	Dennis	Schroder	15	347
2	Devin	Booker	15	331

5.2 Creazione degli indici

Abbiamo definito l'indice per **Analisi_Partita**, in particolare sugli attributi **Data_Partita** e **ID_Membro** per ottimizzare le query 7 e 8:

```
CREATE INDEX idx_Analisi_Partita ON Analisi_Partita(Data_Partita, ID_Membro)
```

L'indice creato è un indice composito su due attributi (Data_Partita e ID_Membro), e utilizza il tipo di indice B-Tree (predefinito in PostgreSQL), particolarmente efficiente per confronti e ordinamenti.

Si è scelto di usare due campi perché migliora l'efficienza della ricerca in diversi casi di query:

- ricerche temporali, quindi in base alla data;
- ricerche solo per un singolo membro, quindi relative esclusivamente al giocatore interessato;
- ricerche che combinano le due precedenti, ottimizzando in termini di efficienza la query.

Così facendo abbiamo eliminato le scansioni complete della tabella, diminuendo il costo di lettura di circa 80% ma aumentando il costo in scrittura 25% poiché ad ogni inserimento, modifica, o eliminazione di una tupla risulta necessario aggiornare l'hash dell'indice, incrementando il tempo necessario per queste operazioni.

Tutto questo è accettabile perché le letture sono molto maggiori delle scritture, quindi vengono migliorate le prestazioni giustificando il costo aggiuntivo tramite il guadagno in efficienza, in più abbiamo flessibilità per le query che usano entrambi gli attributi o uno solo.

6 APPLICAZIONE SOFTWARE

Per connettersi al database dal terminale è fornito del codice C suddiviso in 3 file all'interno della cartella "**codice_C**" chiamati "main.c", "funzioni.h" e "funzioni.c".

È disponibile un Makefile che semplifica l'esperienza dell'utente: eseguito all'interno della cartella 'codice_C', si occupa di compilare il codice sorgente, generare i file oggetto e avviare l'esecuzione del programma tramite il comando:

```
make run
```

Il programma consente inoltre la modifica dei parametri di accesso al database tramite terminale. In caso di connessione riuscita, all'utente verrà mostrato l'elenco delle query disponibili per interagire con il database. Per le query parametriche, l'utente potrà inserire i parametri di interesse, che verranno verificati dal programma per assicurarne la plausibilità prima dell'interrogazione del database.